Erweiterung Raspberry-Pi Ansteuerung des Logistikmodells

Projektrealisierung

für die Prüfung zum

Bachelor of Science

des Studiengangs Wirtschaftsinformatik Application Management

an der

Dualen Hochschule Baden-Württemberg Lörrach

Loïc Dörr, Jacob Ruhnau

31. März 2025

Kurs	WWI22
Ausbildungsfirma	-
Betreuer der Ausbildungsfirma	-
Wissenschaftlicher Betreuer	Dr. Laage-Witt

Ehrenwörtliche Erklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit mit dem Thema

Erweiterung Raspberry-Pi Ansteuerung des Logistikmodells

selbstständig und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel angefertigt habe.

Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten und nicht veröffentlichten Schriften entnommen wurden, sind als solche kenntlich gemacht.

Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt

Die Arbeit ist in gleicher oder ähnlicher Form oder auszugsweise im Rahmen einer anderen Prüfung noch nicht vorgelegt worden.

Lörrach, 31. März 2025

Loïc Dörr Jacob Ruhnau

Hinweis zum Umfang der Arbeit

Der Textteil der vorliegenden Arbeit - beginnend mit der Einleitung bis ausschließlich Quellenverzeichnis - umfasst 11 Seiten.

Inhalt

Ehrenwörtliche Erklärung **II**

Inhalt **III**

1 Einführung und Ziele **1**
1.1 Aufgabenstellung 1
1.2 Qualitätsziele 1
1.3 Stakeholder 2

2 Randbedingungen **2**
2.1 Hardware-Infrastruktur 2
2.2 Kommunikationsprotokoll 2
2.3 Standort und Netzwerk 3
2.4 Projektzeit und Organisation 3

3 Kontextabgrenzung **3**

4 Lösungsstrategie **3**
4.1 Überblick über den elektromechanischen Aufbau 3
4.2 Entwurf der Softwarearchitektur 4
4.3 Implementierung der Steuerungskomponenten 4
4.4 Integration des Hochregallagers 4

5 Komponentensicht **5**
5.1 Logistik Model 5
5.2 high_bay_storage 5
5.3 entry_route 6
5.3.1 Architektur 6
5.3.2 Operationen 6

6 Laufzeitsicht **8**

7 Verteilungssicht **8**

8 Querschnittliche Konzepte **9**
8.1 Raspberry Pis & Interface Board 9
8.2 Pin-Belegungen der originalen SPS-Steuerung 9
8.3 MQTT 9

8.4 Code Versionierung & Zugang zu Github Repo 10
8.5 Dev-Utils 10

9 Architekturentscheidungen 10

10 Qualitätsanforderungen 11

11 Risiken und technische Schulden 11

12 Glossar 11

Anhänge V

1 Einführung und Ziele

Diese Dokumentation basiert auf dem bewährten arc42-Template, das sich zur strukturierten Beschreibung von Software- und Systemarchitekturen etabliert hat. Das Template bietet einen klaren Rahmen zur Dokumentation sowohl technischer als auch konzeptioneller Aspekte eines Systems. Es unterstützt die Nachvollziehbarkeit von Architekturentscheidungen und schafft eine gemeinsame Wissensbasis für alle Beteiligten.
Die Struktur umfasst unter anderem eine Einführung in das System und seine Ziele (Kapitel 1), technische und organisatorische Rahmenbedingungen (Kapitel 2), eine Kontextanalyse (Kapitel 3) sowie detaillierte Sichten auf Architektur (Kapitel 5), Verteilung (Kapitel 6) und Laufzeitverhalten (Kapitel 7). Ergänzt wird dies durch die Dokumentation wichtiger Konzepte, Risiken sowie ein Glossar relevanter Begriffe.

1.1 Aufgabenstellung

Ziel des Gesamtprojekts ist es, das Fischertechnik-Logistikmodell von einer klassischen SPS-Steuerung auf eine IoT-basierte Architektur umzustellen.
In einer ersten Iteration konnte ein vorhergehendes Team 2021 bereits das Hochregallager umstellen. **Im Rahmen dieses Projekts wurde diese Umstellung auf weitere Model-Elemente ausgeweitet, welche zusammen die Waren Zufuhr („Entry Route“) zum Hochregallager ergeben.** Hierbei standen insbesondere der modulare Aufbau, eine einheitliche MQTT-Kommunikation und eine verbesserte Initialisierungs- und Steuerungslogik im Fokus.

1.2 Qualitätsziele

Ziel	Beschreibung
Modularität	Jedes Modul soll unabhängig steuerbar und möglichst wiederverwendbar sein
Erweiterbarkeit	Neue Module und Funktionen sollen ohne tiefgreifende Änderungen integrierbar sein

Ziel	Beschreibung
Robustheit	Die Steuerung soll auch bei fehlerhaften Zuständen kontrolliert abbrechen und nicht hängen bleiben
Wartbarkeit	Der Quellcode soll klar strukturiert, dokumentiert und versioniert sein
Reproduzierbarkeit	Initialisierungsroutinen und Testabläufe sollen nachvollziehbar und zuverlässig funktionieren

Tabelle 1: Qualitätsziele

1.3 Stakeholder

Rolle	Beschreibung
Projekt-Durchführung	Loïc Dörr Jacob Ruhnau
Projekt-Betreuung	Dr. Laage-Witt, Prof. Dr. Olaf
Laborleitung	Ahmed Ayari
DHBW-Github-Adiminstration	Prof. Behrends

Tabelle 2: Stakeholder

2 Randbedingungen

Das Projekt war durch eine Reihe technischer, organisatorischer und infrastruktureller Rahmenbedingungen geprägt.

2.1 Hardware-Infrastruktur

Die bestehende Fischertechnik-Modelllandschaft wurde nicht verändert. Die Steuerung des Models erfolgt über Raspberry Pi Einheiten mit durch Hr. Laage-Witt entwickeltem GPIO-Interface-Boards, welche die Steuerung der Aktoren und Sensoren ermöglicht (vgl. Kapitel 8.1 und 8.2)

2.2 Kommunikationsprotokoll

Die Kommunikation zwischen den Modulen erfolgt über das leichtgewichtige MQTT-Protokoll. Ein Mosquitto-Broker ist auf dem dhbw-raspi-01 eingerichtet (vgl. Kapitel 8.3).

2.3 Standort und Netzwerk

Das Modell wurde im Rahmen des Projekts in das IoT-Labor (Raum S031) umgezogen, um dort eine kontrollierte Entwicklungs- und Testumgebung zu schaffen. Der Raum besitzt ein eigenes WLAN-Netzwerk, der Zugang ist über Hr. Ayari od. Hr. Olaf erfragbar.

2.4 Projektzeit und Organisation

Der Projektzeitraum betrug etwa 4,5 Wochen. Durchgeführt wurde das Projekt als Prüfungsleistung des Moduls „Projektrealisierung“ (W3WI_107.1) im 6. Theoriesemester.

3 Kontextabgrenzung

Das betrachtete Logistikmodell ist ein **in sich geschlossenes, isoliertes System**, das vollständig lokal betrieben und gesteuert wird. Außer der originalen SPS Steuerung, welche ersetzt werden soll, bestehen keine externen produktiven Systemanbindungen oder Abhängigkeiten von Drittsystemen außerhalb der Projektinfrastruktur. Die einzige Kommunikation erfolgt intern über MQTT.

4 Lösungsstrategie

Die Umstellung des Fischertechnik-Logistikmodells auf eine IoT-basierte Architektur erfolgte in dieser Projektiteration schrittweise anhand vier aufeinander aufbauender Phasen:

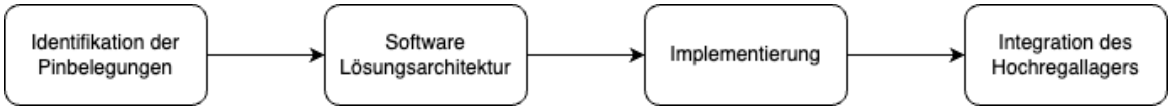

Abbildung 1: Lösungsstrategie

4.1 Überblick über den elektromechanischen Aufbau

Zu Beginn wurde eine detaillierte Analyse des bestehenden Systems durchgeführt. Dies beinhaltete insbesondere die Sichtung und Dokumentation der Pinbelegungen an den Steckerleisten X3, X5 - X6 (Sub-D 37). Da die vom

Hersteller zur Verfügung gestellte Dokumentation nicht dem realen Aufbau entspricht war ein manuelles Mapping von Sensoren und Aktoren sowie die Identifikation Ihrer Pinbelegungen auf den Steckern notwendig. Diese konnten dann auf die verfügbarer GPIO-Pins verteilt werden (vgl. Kapitel 8.2)

4.2 Entwurf der Softwarearchitektur

Die Softwarearchitektur soll so entworfen werden, dass sie modular, wartbar und erweiterbar bleibt. Zentrale Architekturentscheidungen sind in Kapitel 9 beschrieben.

4.3 Implementierung der Steuerungskomponenten

Die Umsetzung soll auf einem weiteren Raspberry Pi samt Interface und über MQTT miteinander gekoppelt stattfinden.

4.4 Integration des Hochregallagers

Das Hochregallager wurde bereits in einem früheren Projekt IoT-fähig gemacht. Im Rahmen dieses Projekts wurde die Anbindung an das Entry Route Modul implementiert.

5 Komponentensicht

5.1 Logistik Model

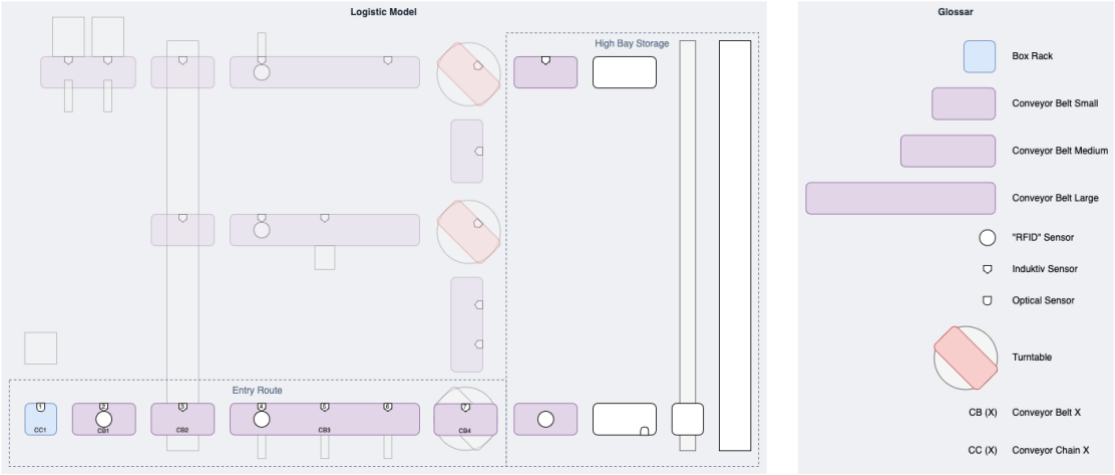

Abbildung 2: Schematische Darstellung des Logistik Models

Das Logistikmodell, basierend auf Fischertechnik, lässt sich in vier abstrahierte Einheiten unterteilen: das Hochregallager (Highbay Storage), die Waren-Eingangsstrecke (Entry Route), die Waren-Ausgangsstrecke (Exit Route) und den mittleren Bereich (Auszeichnung). Diese Einteilung basiert nicht zwingend auf physischen Gegebenheiten und lässt sich nicht direkt auf die Steckverbindungen X1 bis X6 übertragen (vgl. Kapitel 8.2 / Anhang 1), sodass die folgende Implementierung nicht alle Aktoren und Sensoren der jeweiligen abstrahierten Einheit umfasst.

5.2 high_bay_storage

Das Highbay Storage, wie in Abbildung 2, rechts ersichtlich, verfügt über insgesamt 50 Lagerplätze sowie jeweils eine Ein- und Ausgabestation. Im Jahr 2021 wurde es auf eine Python-basierte Steuerung mittels Raspberry Pi umgerüstet. Der implementierte Code bietet verschiedene Operationen zum Ein- und Auslagern, wobei die Kommunikation damals einseitig über das MQTT-Protokoll umgesetzt wurde. Weitere Details finden sich in der Studienarbeit „Aufrüstung des vorhandenen Logistikmodells auf IoT“ (Gruppe 1, DHBW Lörrach, 2021)

5.3 entry_route

Die Entry-Route, dargestellt in Abbildung 2 unten, umfasst alle „Transport-Operatoren“ vom Boxrack bis zum Drehtisch direkt vor der Eingabestation des High-Bay-Storages. Zu den „Transport-Operatoren“ zählen sämtliche Förderbänder, das Boxrack, alle auf der Strecke befindlichen Induktivsensoren sowie die Lichtschranke im Boxrack. Wichtig zu beachten ist, dass der Drehtisch in der aktuellen Applikationslogik zur Vereinfachung als Förderband implementiert wurde. Nicht enthalten sind die RFID-Sensoren sowie die drei Displays.

5.3.1 Architektur

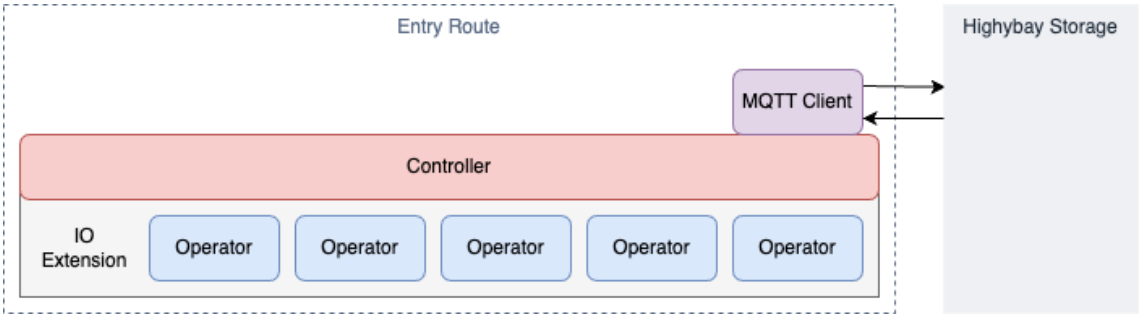

Abbildung 4: Architektur Entry Route

Die Anwendungslogik befindet sich im Controller, während grundlegende Funktionen wie Start, Stopp oder Statusabfragen im Operator implementiert sind. Aktuell existieren die Operatoren **Conveyor Chain, Conveyor Belt** und **Sensor**. Diese sind als Klasse implementiert und können somit einfach definiert werden.
Der Controller subscribed ein MQTT Topic und übergibt dabei eine Callback-Methode, welche bei Eingang einer Nachricht ausgeführt wird. Gleichzeitig kann er über den MQTT-Client auch selbst Nachrichten publizieren.
Die Operatoren sind für die direkte Steuerung der Ports und Pins verantwortlich und nutzen hierfür die IO-Extension.

5.3.2 Operationen

Die Entry Route verfügt aktuell über zwei Operationen: **Run** und **Stop**.

- Die **Run-Operation** transportiert eine Box dem Boxrack zum Highbay Storage und übergibt sie dort.
- Zusätzlich kann der **Run-Operation** ein **Recipe** mitgegeben werden. In diesem Fall wird das Einladen farbiger Kugeln simuliert. Die Simulation beschränkt sich darauf, an der entsprechenden Stelle zu stoppen, da das tatsächliche Einladen aufgrund der Steckerkapazität nicht implementiert werden konnte.
- Die **Stop-Methode** beendet alle Operatoren der Entry Route.

Sowohl die Operationen als auch die Recipes sind durch **Modelklassen** typensicher implementiert und können bei Bedarf erweitert werden. Die Operationen lassen sich über ein JSON-Format ausführen, das dem des **Highbay Storage** entspricht.

```
{ “operation“: “run“ }
{ “operation“: “run“, “recipe“: “Recipe_1“ }
```

Abbildung 5: JSON Format Operation

6 Laufzeitsicht

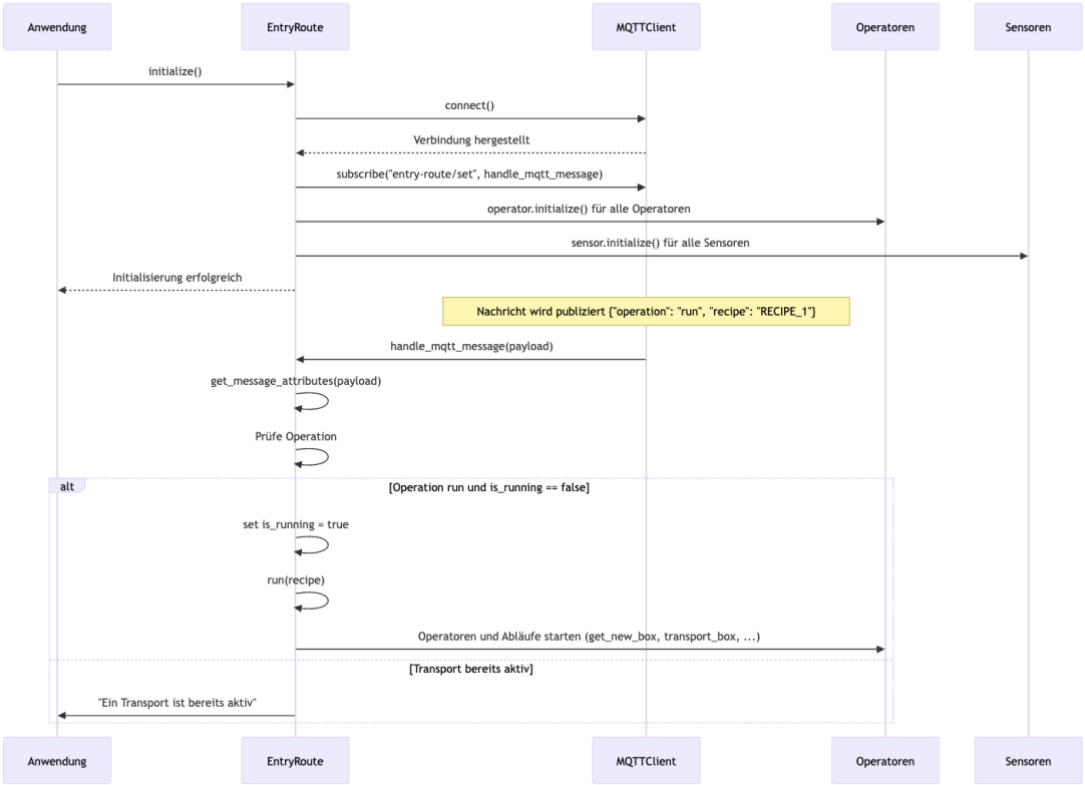

Abbildung 6: Sequenz Diagramm Entry Route

Nachdem das Skript `main.py` ausgeführt wurde, wird die Entry-Route initialisiert. Anschließend subscribt das System das definierte MQTT-Topic und wartet auf eingehende Nachrichten. Sobald eine gültige MQTT-Nachricht über das Topic eintrifft, übernimmt der Controller die Verarbeitung und führt die entsprechende Funktion aus. Während eine Operation läuft, werden weitere eingehende Nachrichten in einer Warteschlange zwischengespeichert, bis die aktuelle Verarbeitung abgeschlossen ist.

7 Verteilungssicht

Dieses Kapitel wird zum aktuellem Zeitpunkt nicht benötigt.

8 Querschnittliche Konzepte

8.1 Raspberry Pis & Interface Board

Die Steuerung erfolgt über Raspberry Pi Einplatinencomputer, die jeweils mit einem speziell entwickelten GPIO-Interface-Board ausgestattet sind. Das Interface-Board sorgt über Optokoppler für die notwendige galvanische Trennung zwischen Steuerlogik und der Aktor-/Sensorebene. Um eine einfache Integration im Code zu gewährleisten, wurde eine GPIO-Extension von Herrn Laage-Witt bereitgestellt (vgl. die Studienarbeit „Aufrüstung des vorhandenen Logistikmodells auf IoT“ der Gruppe 1, 2021). Im Rahmen dieser Projektiteration wurde Sie als Singleton adaptiert und sorgt dafür, dass die Hardware-Ansteuerung nur einmal initialisiert wird und in allen Operatoren verfügbar ist.

8.2 Pin-Belegungen der originalen SPS-Steuerung

Da die ursprünglich vorhandene Dokumentation der SPS-Pinbelegung nicht mit dem realen Aufbau übereinstimmte wurde im Rahmen dieser Projektiteration eine manuelle Analyse durchgeführt. Diese beinhaltete die systematische Durchmessung und Zuordnung der Sensoren und Aktoren zu den jeweiligen Pins der Sub-D-Stecker (z. B. X5 für die Entry Route). Die Ergebnisse wurden tabellarisch in einer Excel-Datei dokumentiert (siehe Anhang 1). Diese bildet die Grundlage für die GPIO-Zuordnung am Interface Board.
Die Steuerung des SPS erfolgte hierbei über sechs Stecker, wobei zwei für das Hochregallager (dhbw-raspi-01) entfallen (X1, X2). Der restliche Teil des Models erfolgt über 4 weitere Stecker (X3-X5). Der dhbw-raspi-02 belegt X3 und X5, wobei nicht alle Aktoren und Sensoren angesteuert werden können, da das Interfaceboard in der aktuellen Version über 16 Ausgänge (zwei Ports â 9 Pins) und 32 Eingänge (vier Ports â 8 Pins) verfügt. Eine Erweiterung durch Hr. Laage-Witt wäre möglich.

8.3 MQTT

Zur Kommunikation zwischen den Modulen wird das MQTT-Protokoll eingesetzt. Die einzelnen Module (z. B. Entry Route und Hochregallager) publishen und subscriben auf dedizierten Topics. Das Nachrichtenformat ist JSON-basiert und

transportiert sowohl Steuerbefehle (z. B. "Run", "Stop") (vgl. Kapitel 5.2.3) als auch Zusatzinformationen wie "Recipe"-Parameter. Der MQTT-Client der Entry Route wurde mit Callback-Funktionen versehen, um eine flexible und modulare Reaktion auf empfangene Nachrichten zu ermöglichen.

8.4 Code Versionierung & Zugang zu Github Repo

Der gesamte Quellcode wurde in einem privaten Repository auf GitHub versioniert, das über die DHBW durch Prof. Behrends verwaltet wird. Für die Raspberry Pis wurden Personal Access Token (PAT) hinterlegt, um einen reibungslosen Pull/Push-Zugriff zu ermöglichen. Der PAT hat eine Lebenszeit von einem Jahr, kann aber durch Prof. Behrends erneuert werden.
Änderungen wurden über Branches entwickelt und regelmäßig in den Main Branch gemergt. Die Dokumentation im Repository wurde parallel zur technischen Umsetzung gepflegt und enthält u. a. Hinweise zur Installation, Konfiguration sowie ein ReadMe zur Modulstruktur.

8.5 Dev-Utils

Für Entwickler stehen spezielle Modi zur Verfügung, um einzelne Komponenten zu testen oder den Sensorzustand direkt auszugeben. Hierzu muss die entry_route/main.py aus dem Terminal ausgeführt werden mit entweder „devs“ oder „devc“ und ergänzt um die Parameter Port, Pin, Dauer in Sekunden.

9 Architekturentscheidungen

Entschied	Beschreibung
Trennung von Logik und Infrastruktur	Steuerlogik in Controller-Klassen, Kommunikation über dedizierte MQTT-Clients
Wiederverwendbarkeit durch Abstraktion	Einbindung von Sensoren und Aktoren über eigene Klassen mit Initialisierungs- und Kontrollfunktionen
Robustheit	Einführung einer Initialisierungsroutine („Referenzfahrt“) und Fehlererkennung durch Timeouts
IO Extension	Verwendung eines Singleton-Musters für GPIO-Ansteuerung

Entschied	Beschreibung

Tabelle 3: Hier steht die Beschriftung der Tabelle

10 Qualitätsanforderungen

Dieses Kapitel wird zum aktuellem Zeitpunkt nicht benötigt.

11 Risiken und technische Schulden

Super-Controller über allem Modulen; Handler / Orchestrator

Aktuell nicht notwendig bei nur zwei Modulel

12 Glossar

Dieses Kapitel wird zum aktuellem Zeitpunkt nicht benötigt.

Anhänge

Anhang 1: „Pin Übersicht & Mapping.xlsx“

Dokumentation der Ausgelesenen Pin Belegungen

Anhang 2: „Video-Kurz-Dokumentation IoT-Logistikmodel.mp4“

Video Dokumentation

Anhang 2: Code.zip

Der Code beider Module als Zip-Datei.